

Démarche de développement du logiciel

Projet SmokeWatch — logiciel de supervision vidéo intelligente pour site cimentier, exploitant des modèles de détection **déjà entraînés** (fumée, feu, non-port des EPI).

Ce document décrit la conduite du développement : phases, entrées, sorties, et critère qui autorise à passer à la suite. Il ne traite pas de la constitution du jeu de données ni de l'entraînement des modèles, considérés comme acquis.

Vue d'ensemble

Phase	Objet	Semaine	Verrou de sortie
0	Cadrage fonctionnel	S1	Cahier des charges validé
1	Caractérisation des modèles fournis	S1	Fiche technique par modèle
2	Conception et contrats d'interface	S2	Interfaces figées
3	Développement incrémental	S2-S8	7 incréments démontrés
4	Intégration et tests	S8	Tests de charge et robustesse passés
5	Réglage et validation sur site	S9	Taux de fausses alertes mesuré
6	Déploiement et documentation	S9-S10	Système en service, manuels livrés

Phase 0 — Cadrage fonctionnel

But : transformer une demande floue en objectifs mesurables.

Questions à trancher avec le service HSE et la maintenance :

- **Quelles caméras ?** On ne couvre pas tout le site. Priorité : four, broyeur, stockage combustible, convoyeurs, ensachage. Établir une liste ordonnée.
- **Que se passe-t-il après l'alerte ?** Un opérateur en salle de contrôle, une notification sur téléphone, une remontée vers l'automate ? Sans destinataire identifié, le logiciel ne sert à rien.
- **Quel niveau de fausses alertes est tolérable ?** Au-delà de **2 par jour et par caméra**, l'expérience montre que le système est ignoré, puis éteint.

- **Quel délai de détection acceptable ?** Cible réaliste : **moins de 15 s** entre le début du phénomène et l'alerte reçue.
- **Quelles contraintes réseau ?** Les flux peuvent-ils sortir du VLAN industriel ? Le serveur peut-il envoyer des e-mails vers l'extérieur ?
- **Quel matériel disponible ?** Serveur avec ou sans GPU, espace disque, droits d'installation. Cette réponse conditionne le nombre de caméras tenables.

Livrables : cahier des charges, liste priorisée des caméras avec zone et modèles à y appliquer, objectifs chiffrés.

Piège : accepter un périmètre du type « détecter tout ce qui est dangereux ». Un périmètre non borné est la première cause d'échec d'un projet de fin d'études.

Phase 1 — Caractérisation des modèles fournis

Les modèles existent, mais on ne peut pas concevoir le logiciel autour d'eux sans savoir **ce qu'ils coûtent et ce qu'ils savent faire**. Cette phase remplace l'entraînement.

1.1 Inventaire technique

Pour chaque modèle, établir une fiche :

Élément	Pourquoi c'est nécessaire
Liste exacte des classes de sortie	l'interface et la base doivent les nommer correctement
Résolution d'entrée attendue	conditionne le redimensionnement dans la boucle
Format des poids et version du cadriciel	risque d'incompatibilité de version
Latence par image en CPU et en GPU	détermine le nombre de caméras tenables
Empreinte mémoire vidéo	détermine combien de modèles cohabitent sur une carte

1.2 Calibration des seuils

Un modèle livré avec un seuil par défaut n'est jamais réglé pour le site. Il faut un **petit jeu de validation issu des caméras réelles** — quelques centaines d'images suffisent, annotées simplement en présence/absence.

Faire varier le seuil de confiance de 0,3 à 0,8 et relever, pour chaque valeur, la précision et le rappel. Le seuil retenu par zone est celui qui donne le rappel visé au taux de

fausses alertes acceptable. Une zone poussiéreuse demandera un seuil plus élevé qu'une zone propre — d'où un seuil **par caméra**, pas un seuil global.

1.3 Vérification de l'écart au terrain

Vérifier que les modèles se comportent correctement sur le flux réel : même compression, même résolution, mêmes angles. Un modèle validé sur images nettes et appliqué à un flux 720p compressé perd sensiblement en performance.

Livrables : fiche technique par modèle, tableau des seuils par caméra, tableau de latences servant au dimensionnement.

Verrou : si un modèle s'avère inexploitable sur le flux réel, il faut le savoir en semaine 1, pas en semaine 8.

Phase 2 — Conception et contrats d'interface

But : permettre à quatre personnes de développer en parallèle sans se bloquer.

2.1 Architecture retenue

Cinq couches indépendantes :

1. **Acquisition** — un thread léger par caméra, ne conservant que la dernière image et un tampon de pré-roll.
2. **Inférence** — une boucle unique regroupant les images de toutes les caméras partageant un modèle, en un seul appel par lot.
3. **Règles** — zone d'intérêt, confirmation temporelle, cooldown.
4. **Persistance et diffusion** — base des événements, clips, API, WebSocket.
5. **Interface** — dashboard de supervision.

Deux décisions structurantes à justifier dans le mémoire :

- **Une seule boucle d'inférence, pas une par caméra.** L'approche naïve lance un thread complet par caméra ; à partir de 5 ou 6 flux, les threads se disputent le GPU et chacun charge sa propre copie du modèle. Ici, le coût dominant devient le nombre d'appels au GPU, pas le nombre de caméras.
- **On n'analyse pas 25 images par seconde.** Quatre suffisent : la fumée s'installe sur plusieurs secondes. Analyser à pleine cadence consomme six fois plus de calcul sans détecter plus tôt.

2.2 Contrats à figer

Avant tout développement parallèle : format de l'objet `Detection`, format de l'objet `Alert`, schéma de la base, schémas JSON de l'API et du WebSocket, structure du fichier de configuration.

Tant qu'un contrat n'est pas livré, chacun développe contre un **bouchon**. Les bouchons servent ensuite de base aux tests automatisés — ils ne sont pas du travail perdu.

Livrables : document d'architecture, contrats d'interface, squelette de projet compilable avec bouchons en place.

Phase 3 — Développement incrémental

Chaque incrément est **démontrable**. Un incrément non démontré n'est pas terminé, quel que soit l'avancement déclaré.

N°	Contenu	Ce qu'on démontre	Semaine
I1	Lecture d'un flux RTSP, affichage, reconnexion	une caméra à l'écran, câble débranché puis rebranché	S2
I2	Inférence sur ce flux, boîtes dessinées	détection visible en direct	S3
I3	Multi-caméras, inférence par lot	8 flux simultanés, latence affichée	S4
I4	Moteur de règles, journalisation en base	alertes filtrées, poussière ignorée	S5
I5	Snapshots, clips, notifications	preuve reçue par e-mail avec image	S6
I6	API, dashboard, alertes temps réel	supervision complète dans le navigateur	S7
I7	Tracé de ROI, seuils à chaud, rapports	réglage sans redémarrage du service	S8

Pratiques minimales

- Branche par rôle, fusion hebdomadaire dans la branche principale, revue croisée par une personne d'une autre couche.
 - **Tests unitaires obligatoires sur deux modules** : le moteur de règles et la couche de persistance. Ce sont les deux endroits où un défaut est silencieux — une alerte qui n'est jamais levée ne provoque aucune erreur visible.
 - Journalisation systématique, avec niveau configurable. En exploitation, le journal est le seul moyen de comprendre pourquoi une alerte n'est pas partie.
 - Aucune donnée sensible dans le dépôt : identifiants RTSP et mots de passe SMTP restent dans le fichier de configuration, exclu du suivi de version.
-

Phase 4 — Intégration et tests

4.1 Tests fonctionnels sur séquences rejouées

Remplacer les flux caméra par des fichiers vidéo enregistrés. Même entrée, même sortie attendue : c'est ce qui rend les résultats **reproductibles** et donc publiables dans le mémoire.

Constituer une bibliothèque de séquences de référence :

Séquence	Attendu
Départ de fumée réel	alerte levée en moins de 15 s
Passage de camion soulevant de la poussière	aucune alerte
Panache de vapeur du refroidisseur	aucune alerte
Brume matinale	aucune alerte
Ouvrier sans casque en zone d'ensachage	alerte de gravité moyenne
Flux interrompu en cours de séquence	caméra marquée hors ligne, reprise

4.2 Tests de charge

Mesurer pour 4, 8, 12 et 16 caméras : latence de boucle, occupation GPU, mémoire, images analysées par seconde. Tracer la courbe et identifier le point de rupture. **Cette courbe justifie le dimensionnement matériel recommandé** et constitue un résultat chiffré du mémoire.

4.3 Tests de robustesse

Provoquer délibérément les pannes :

Panne simulée	Comportement attendu
Câble caméra débranché	reconnexion automatique, caméra marquée hors ligne
Coupure réseau de 5 minutes	reprise sans redémarrage du service
Disque plein	événements toujours enregistrés en base, clips ignorés
Serveur de messagerie injoignable	alerte en base, échec de notification journalisé
Redémarrage du serveur	reprise automatique du service
Deux alertes simultanées sur deux caméras	les deux traitées, aucune perdue

Un système de sécurité qui s'arrête silencieusement est plus dangereux qu'une absence de système : l'exploitant croit être couvert.

Livrables : rapport de tests, courbe de montée en charge, liste des défauts corrigés.

Phase 5 — Réglage et validation sur site

5.1 L'expérience centrale du projet

Les modèles étant fournis, **la contribution technique du projet est la chaîne de traitement qui les rend exploitables**. Il faut la mesurer.

Rejouer les mêmes séquences de référence dans quatre configurations :

Configuration	Zone d'intérêt	Confirmation temporelle	Cooldown
C0 — modèle brut	non	non	non
C1	oui	non	non
C2	oui	oui	non
C3 — système complet	oui	oui	oui

Relever pour chacune : nombre total d'alertes, dont fausses, taux de détection des incidents réels, délai moyen de détection.

Le résultat attendu est une chute importante des fausses alertes de C0 à C3, avec une perte faible sur le taux de détection et quelques secondes de délai supplémentaire. Ce

tableau est le résultat le plus fort du projet : il démontre que le travail d'ingénierie autour du modèle vaut autant que le modèle lui-même — argument d'autant plus important que les modèles ne sont pas de votre fait.

5.2 Réglage par zone

Ajuster caméra par caméra : seuil de confiance, polygone de zone d'intérêt, nombre d'images de confirmation, durée de cooldown. Consigner chaque valeur retenue **et sa justification** — c'est une annexe du mémoire.

5.3 Semaine de fonctionnement réel

Laisser tourner sept jours sans intervention. Relever chaque jour : alertes levées, alertes justifiées, incidents manqués s'il y en a, incidents techniques.

5.4 Retour des opérateurs

Faire utiliser le dashboard par les opérateurs de la salle de contrôle. Ce qui remonte le plus souvent : trop d'informations à l'écran, alerte pas assez visible, clip difficile à retrouver. Ces retours comptent dans l'évaluation d'un projet industriel.

Livrables : tableau comparatif C0-C3, journal de la semaine, tableau des réglages par caméra, synthèse des retours utilisateurs.

Phase 6 — Déploiement et documentation

6.1 Mise en service

- Installation en service système avec redémarrage automatique.
- Sauvegarde périodique de la base des événements.
- Purge automatique selon la durée de rétention configurée.
- Supervision minimale : vérifier que le service tourne et que les caméras sont connectées.

6.2 Documentation

Document	Destinataire
Manuel d'installation	service informatique
Manuel opérateur	salle de contrôle
Documentation de l'API	maintenance et évolutions
Document d'architecture	mémoire, chapitre conception

6.3 Structure du mémoire

1. Contexte industriel — le site, les risques, l'existant
2. Modèles fournis : caractérisation et calibration — phase 1
3. Analyse des besoins et cahier des charges — phase 0
4. Architecture du logiciel et choix de conception — phase 2
5. Réalisation, incrément par incrément — phase 3
6. Traitement des fausses alertes — phases 3 et 5, **le chapitre à soigner**
7. Tests, performance et montée en charge — phase 4
8. Validation sur site et résultats — phase 5
9. Limites et perspectives

6.4 Démonstration

Prévoir **deux versions** : une en direct sur les caméras, une sur séquences enregistrées. La démonstration en direct dépend du réseau et d'un incident qui ne se produira pas au bon moment ; la version enregistrée garantit que le jury voit une détection réelle.

Indicateurs de réussite

Deux familles, à ne pas confondre.

Indicateurs techniques (ce que mesure l'équipe) :

Indicateur	Cible
Caméras traitées simultanément	≥ 8
Latence de boucle	$< 1 \text{ s}$
Latence image capturée $\rightarrow$ notification	$< 3 \text{ s}$
Disponibilité du service sur la semaine de test	$> 99 \%$

Indicateurs d'exploitation (ce qui décide de l'adoption) :

Indicateur	Cible
Taux de détection des incidents réels	$> 90 \%$
Fausses alertes par caméra et par jour	< 2
Délai de détection	$< 15 \text{ s}$
Taux d'acquiescement des alertes par les opérateurs	$> 80 \%$

Le dernier indicateur est le plus révélateur : si les opérateurs cessent d'acquiescer, le système a perdu leur confiance, quels que soient les scores techniques.

Risques et parades

Risque	Impact	Parade
Accès réseau aux caméras retardé	bloque le développement dès S2	banc de test sur fichiers vidéo dès S1
Modèles trop lourds pour le matériel	nombre de caméras réduit	mesurer en S1, réduire cadence ou résolution
Modèles inadaptés au flux réel	remet en cause le projet	vérification en S1, retour vers l'équipe modèles
Pas de GPU disponible	temps réel impossible	décision matérielle en S1, repli sur 2 caméras en CPU
Fausse alertes irréductibles	rejet par l'exploitant	resserrer les zones, réserver aux zones peu poussiéreuses
Contrats d'interface non figés	quatre personnes bloquées	réunion dédiée en fin de S2, non reportable
Dérive du périmètre	projet inachevé	périmètre figé en S1, nouvelles demandes en « perspectives »

Ce qui distingue un bon projet

Les modèles étant fournis, le jury évaluera surtout **l'ingénierie logicielle**. Un bon projet montre :

- une architecture justifiée par des mesures, pas par des préférences ;
- une courbe de montée en charge qui dimensionne le matériel ;
- une mesure chiffrée de l'apport de chaque filtre anti-fausse alertes ;
- des tests de robustesse sur pannes réelles, pas seulement des cas nominaux ;
- des résultats reproductibles grâce au banc de rejeu ;
- des limites énoncées honnêtement.

Ce dernier point compte plus qu'on ne le croit. Un jury fait davantage confiance à une équipe qui explique dans quelles conditions son système échoue qu'à une équipe qui annonce un fonctionnement parfait.